

# Computer vision

## Image filtration

# Filtration

## Linear methods

*in spatial domain* – using convolution  
-> direct manipulation of pixels in an image

*in frequency domain*  
-> using Fourier transform (or Cosine transform...or other linear transform)

## Non-linear (other) filtering methods

ordering filters...

# Ordering filtering methods

Non-linear method

# 2-D order-statistic filtering

## Order filter

Special cases: minimum filter, maximum filter, median filter

### Steps:

1. Define the neighbourhood around the pixel
2. Sort values of this neighbourhood pixels in the numerical order
3. Replaces each element in the output image by the  $order^{th}$  element in the sorted set of neighbours.

# 2-D median filter

The median filter is a non-linear digital filtering technique, often used to remove noise from images or other signals.

(special case of the order-statistic filtering)

Steps:

1. Define the neighbourhood around a pixel
2. Sort values of this neighbourhood in the numerical order
3. Pick the median of the sorted set as the new pixel value

# Median filter - Example

Median filter 5x5



...reduce noise and preserve (only partially!) edges

# Image linear filtration Convolution

# Convolution

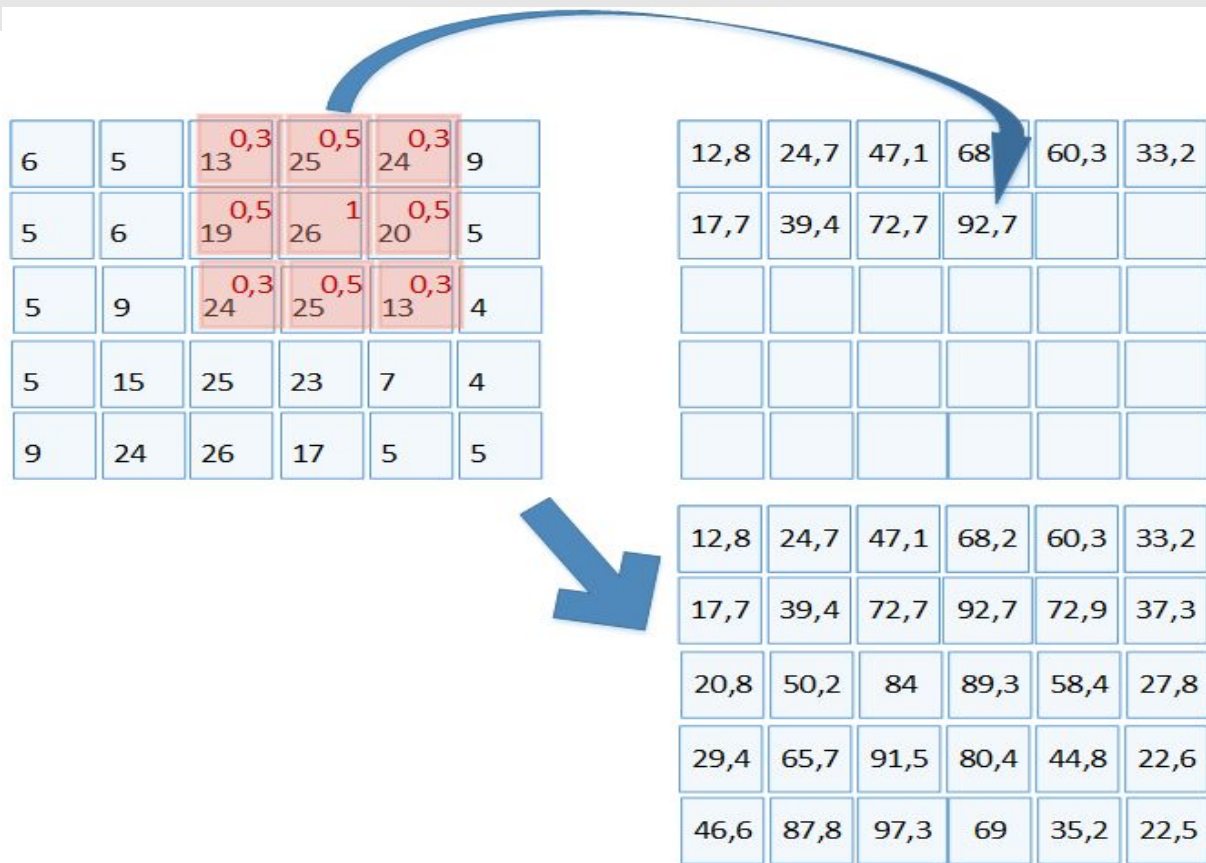
## Linear operation

The discrete convolution of two functions  $f(x, y)$  and  $g(x, y)$  of size  $M \times N$  is denoted by  $f(x, y) * g(x, y)$  and is defined by the expression

$$f(x, y) * g(x, y) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) g(x - m, y - n)$$

Where the function  $f(x, y)$  is an image  
 $g(x, y)$  is a filter kernel

# Example of 2D Convolution



# Dot product (skalárny súčin)

The dot product of two vectors  $\mathbf{a} = [a_1, a_2, \dots, a_n]$  and  $\mathbf{b} = [b_1, b_2, \dots, b_n]$  is defined as:

$$\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

...the sum of the products of the corresponding coordinates

# Border Effects

- **zero:** set all pixels outside the source image to 0
- **constant:** set all pixels outside the source image to a specified border value
- **mirror:** reflect pixels across the image edge
- **reduced size:** size of output image differ from size of input image

# Properties of the Convolution

Commutative:

$$f * g = g * f$$

Associative:

$$f * (g_1 * g_2) = (f * g_1) * g_2$$

Distributive:

$$f * (g_1 + g_2) = f * g_1 + f * g_2$$

Scaling :

$$f * (k.g) = k.(f * g)$$

$k$  - const.

Shift invariant:

Operator behaves the same everywhere,

*i.e.* the value of the output depends on the pattern in the image neighborhood, not the position of the neighborhood.

# 2D cross-correlation

The mathematical calculation of cross-correlation is similar as convolution in time domain.

->this is the same as convolution using flipped filter

If the filter is symmetric then the output of both the expression would be same.

$$\text{Corr}(f(x, y) g(x, y)) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n) g(x + m, y + n)$$

# 2D cross-correlation vs. convolution

$$\text{Corr}(f(x, y), g(x, y)) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n)g(x + m, y + n)$$

Note the difference!

$$f(x, y) * g(x, y) = \sum_{m=0}^{M-1} \sum_{n=0}^{N-1} f(m, n)g(x - m, y - n)$$


**Correlation** is mainly used for expression of similarity between two signals/sequences.

**Convolution** is mainly used for linear filtering.

OpenCv:

cross-correlation:

`matchTemplate()`

parameter: `TM_CCORR`

convolution:

`filter2D()`

`cuda::convolve()`

# 2D cross-correlation vs. convolution

Convolution is preferred mathematically as it is associative,

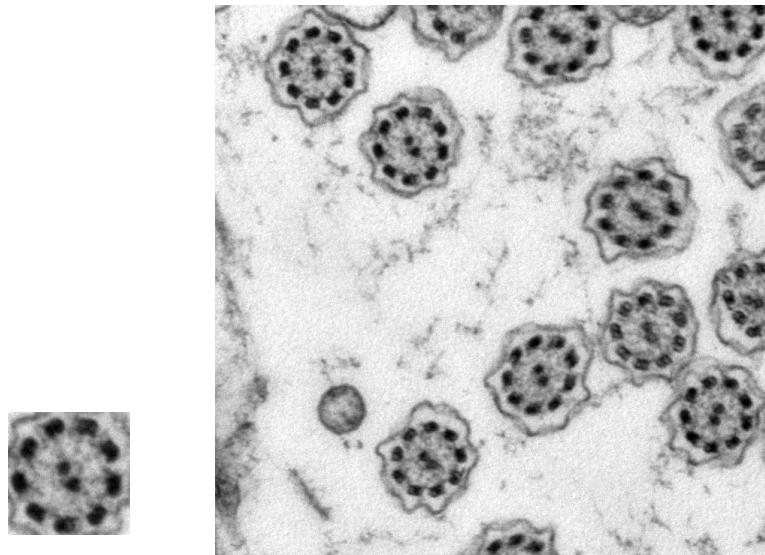
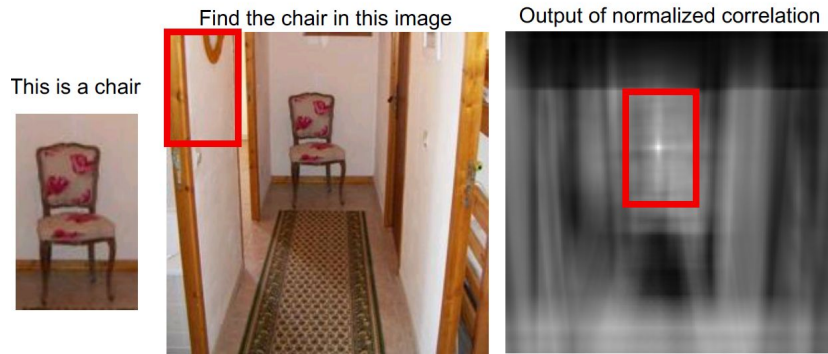
$$(x * h) * h = x * (h * h)$$

- Correlation is not associative,

$$(x \otimes h) \otimes h \neq x \otimes (h \otimes h)$$

- Correlation preferred, however, for signal matching/detection.

# Object detection using correlation



# Filtering using convolution

## Smooth filters

# Mean (blur) filter

## Smooth filter

Convolution kernel 3x3:

$1/9$

|   |   |   |
|---|---|---|
| 1 | 1 | 1 |
| 1 | 1 | 1 |
| 1 | 1 | 1 |

Convolution kernel 5x5:

$1/25$

|   |   |   |   |   |
|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |
| 1 | 1 | 1 | 1 | 1 |

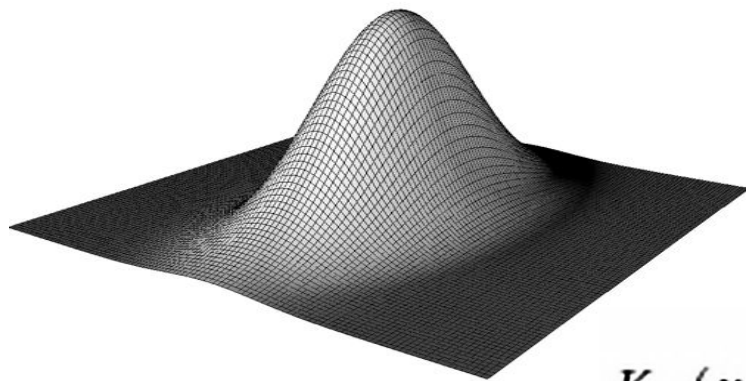
# Mean filter



# Gauss filter

## Smooth filter

Gauss convolution kernel 5x5 (integer):


$$\frac{1}{273}$$

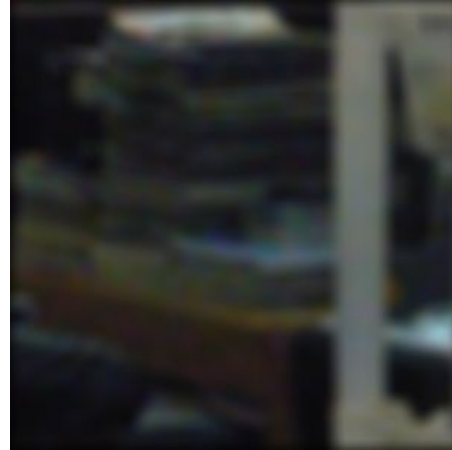
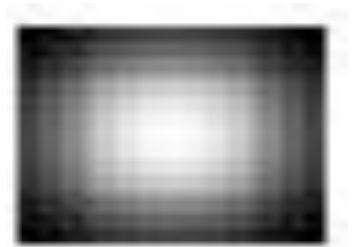
|   |    |    |    |   |
|---|----|----|----|---|
| 1 | 4  | 7  | 4  | 1 |
| 4 | 16 | 26 | 16 | 4 |
| 7 | 26 | 41 | 26 | 7 |
| 4 | 16 | 26 | 16 | 4 |
| 1 | 4  | 7  | 4  | 1 |

$$K_{\sigma}(x, y) := \frac{1}{2\pi\sigma^2} \cdot \exp\left(\frac{-x^2 - y^2}{2\sigma^2}\right)$$

# Example of 2D Convolution

## Gaussian filter kernel

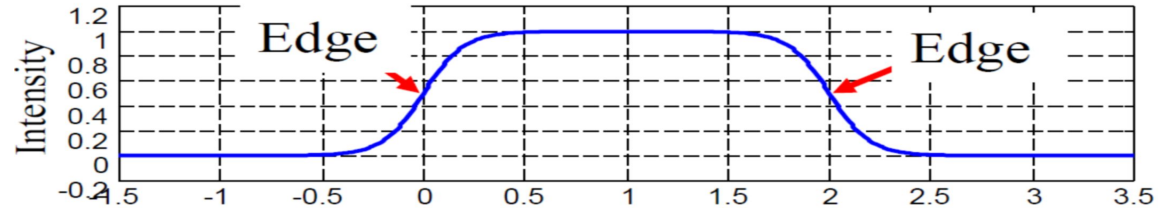
Gaussian filter kernel size:15x15



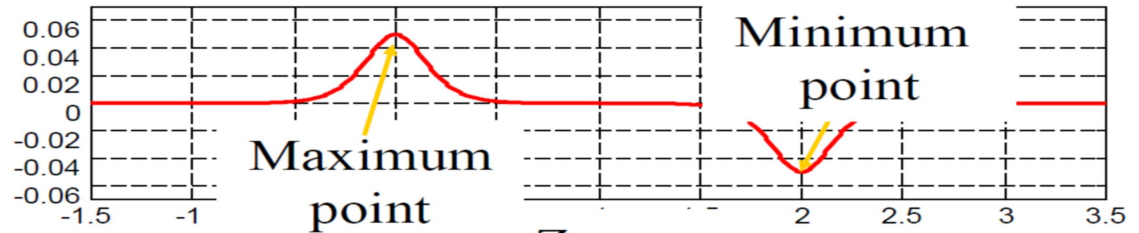
# Image linear filtration- Edge filters

# Edge detection

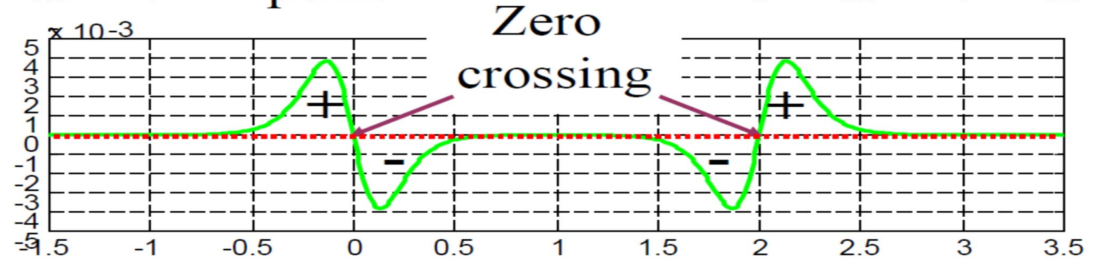
Gray level profile line



The 1-st derivative



The 2-nd derivative



# Prewitt filter

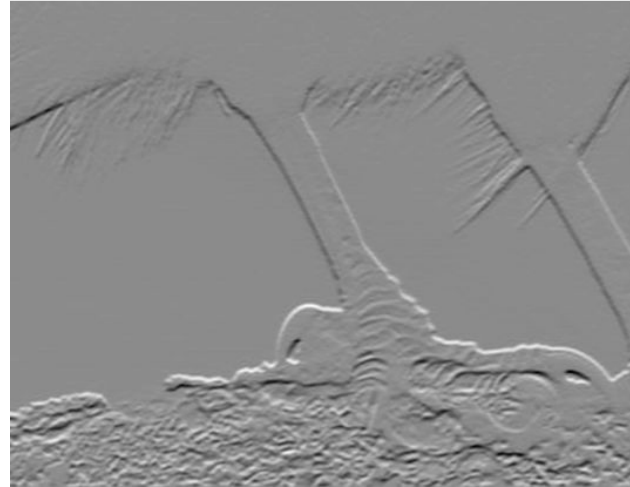
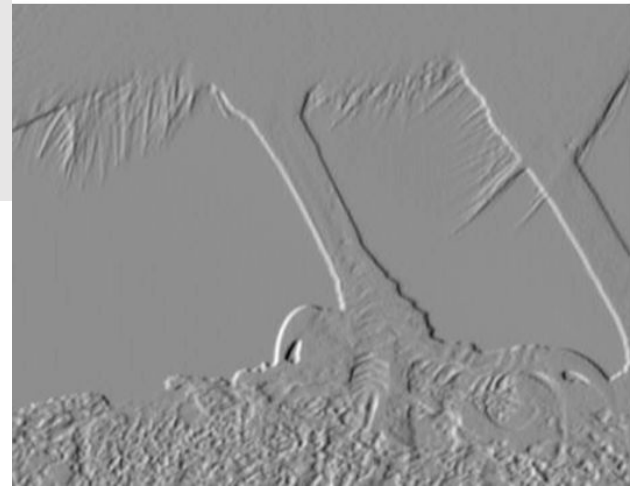
Gradient edge filter (1-st deriv.)

Vertical convolution kernel:

$$\begin{bmatrix} 1 & 1 & 1 \\ 0 & 0 & 0 \\ -1 & -1 & -1 \end{bmatrix}$$

Horizontal convolution kernel:

$$\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

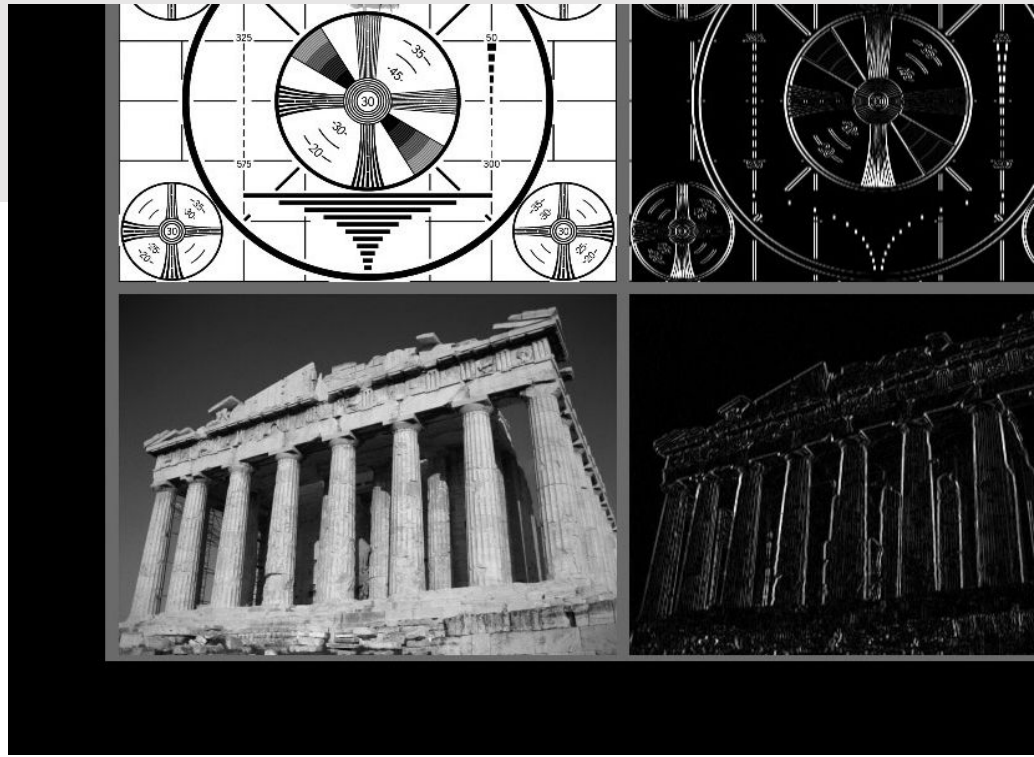


# Sobel filter

Gradient edge filter  
(1-st deriv.)

Sobel convolution  
kernels:

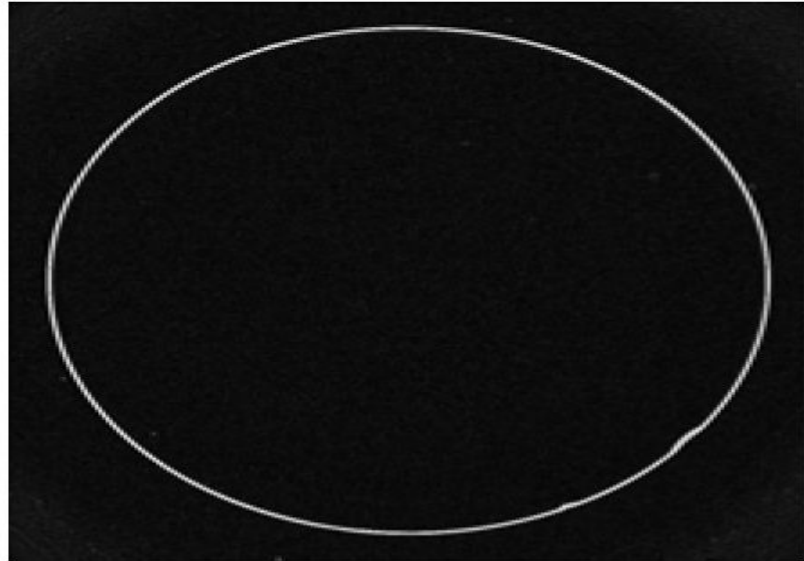
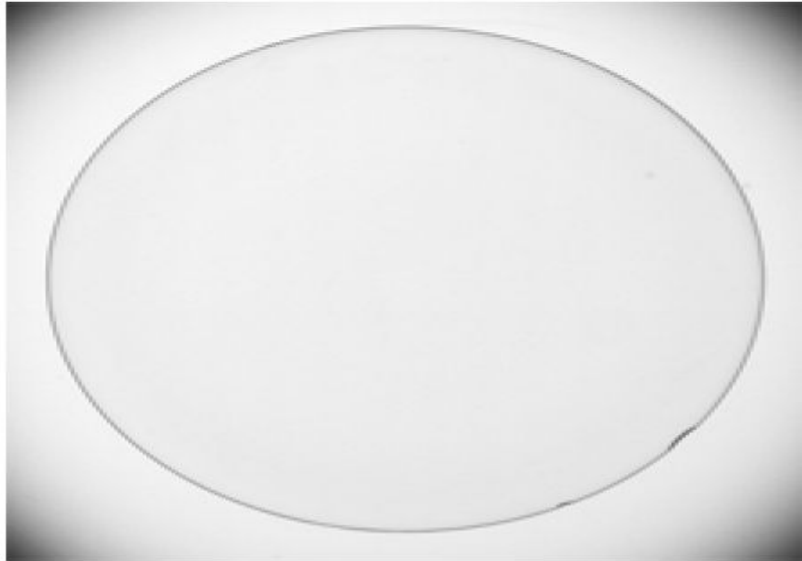
$$\mathbf{G}_y = \begin{bmatrix} +1 & +2 & +1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} * \mathbf{A} \quad \text{and} \quad \mathbf{G}_x = \begin{bmatrix} +1 & 0 & -1 \\ +2 & 0 & -2 \\ +1 & 0 & -1 \end{bmatrix} * \mathbf{A}$$



# Use of First Derivatives for Enhancement - The Sobel Gradient

(a) Optical image of contact lens (note defects on the boundary at 4 and 5 o'clock).

(b) Sobel gradient.



# Roberts filter

Gradient edge filter (1-st deriv.)

Roberts convolution kernels:

$$h_1 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix} \quad h_2 = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

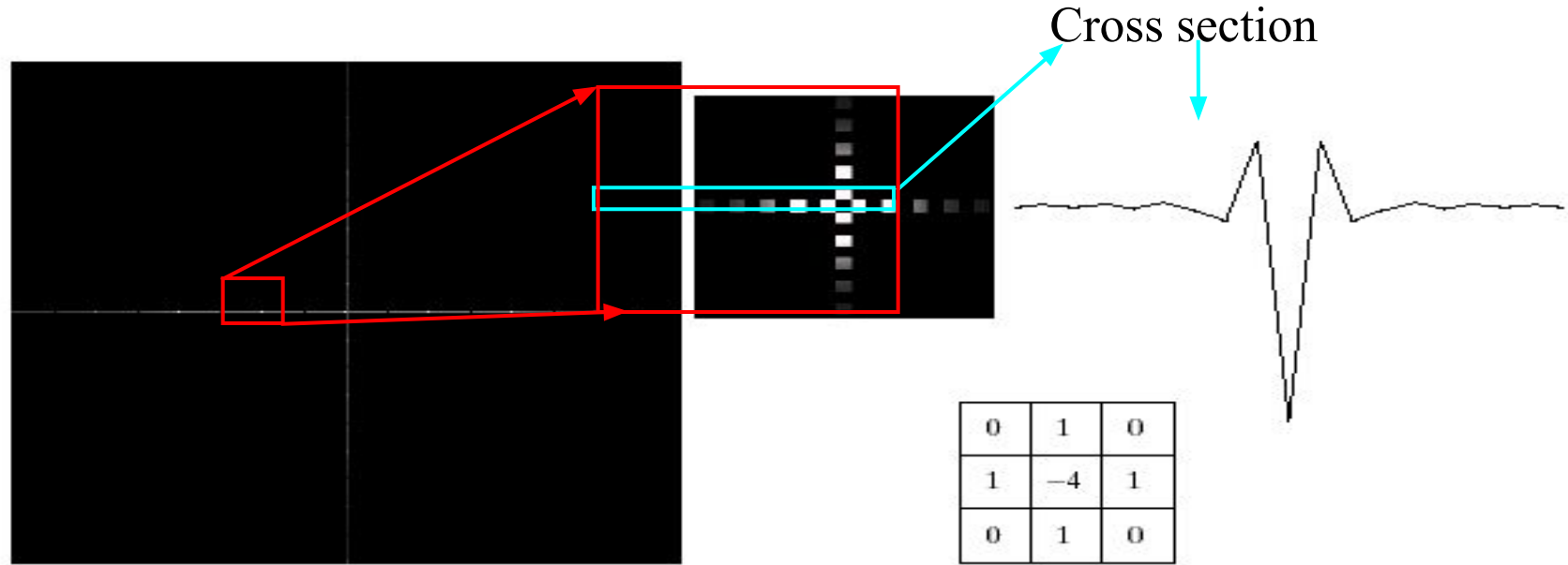
# Laplacian filter

## Edge filter (2nd derivative)

Laplacian for a function (image)  $f(x, y)$  of two variables is defined as:

$$\nabla^2 u = \frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} = 0$$

# Laplacian Filter



# Laplacian convolution kernel (filter mask )

Mask includes/or not includes also the diagonal neighbors.

convolutional kernels 3x3:

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 0  | 1  | 0  | 1  | 1  | 1  |
| 1  | -4 | 1  | 1  | -8 | 1  |
| 0  | 1  | 0  | 1  | 1  | 1  |
| 0  | -1 | 0  | -1 | -1 | -1 |
| -1 | 4  | -1 | -1 | 8  | -1 |
| 0  | -1 | 0  | -1 | -1 | -1 |

convolutional kernel 5x5:

$$\begin{bmatrix} 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 2 & 1 & 0 \\ 1 & 2 & -16 & 2 & 1 \\ 0 & 1 & 2 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 \end{bmatrix}$$

# Gabor filters

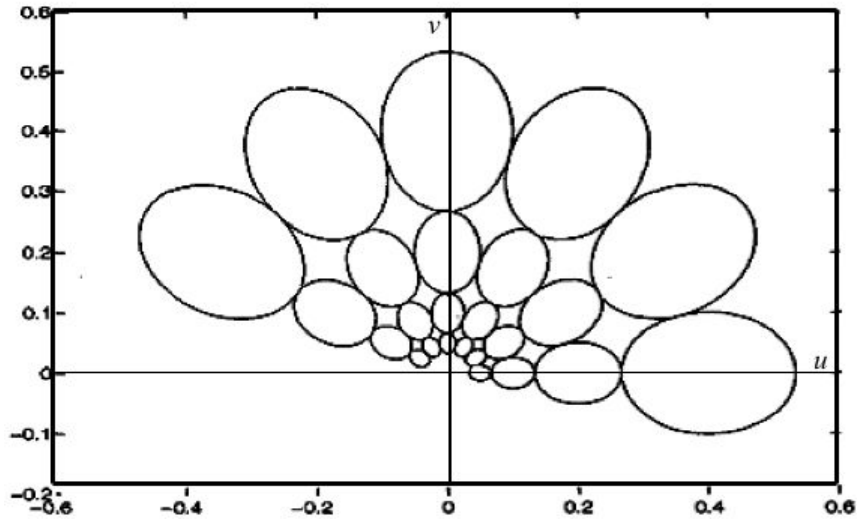
# Gabor Filters

Gabor wavelets

Wavelets at different frequencies and different orientations

$$\gamma(x,y) = \frac{1}{2\pi\sigma_x\sigma_y} \exp\left[-\frac{1}{2}\left(\frac{(x-x_0)^2}{\sigma_x^2} + \frac{(y-y_0)^2}{\sigma_y^2}\right) + 2\pi\sqrt{-1}(u_0x + v_0y)\right]$$

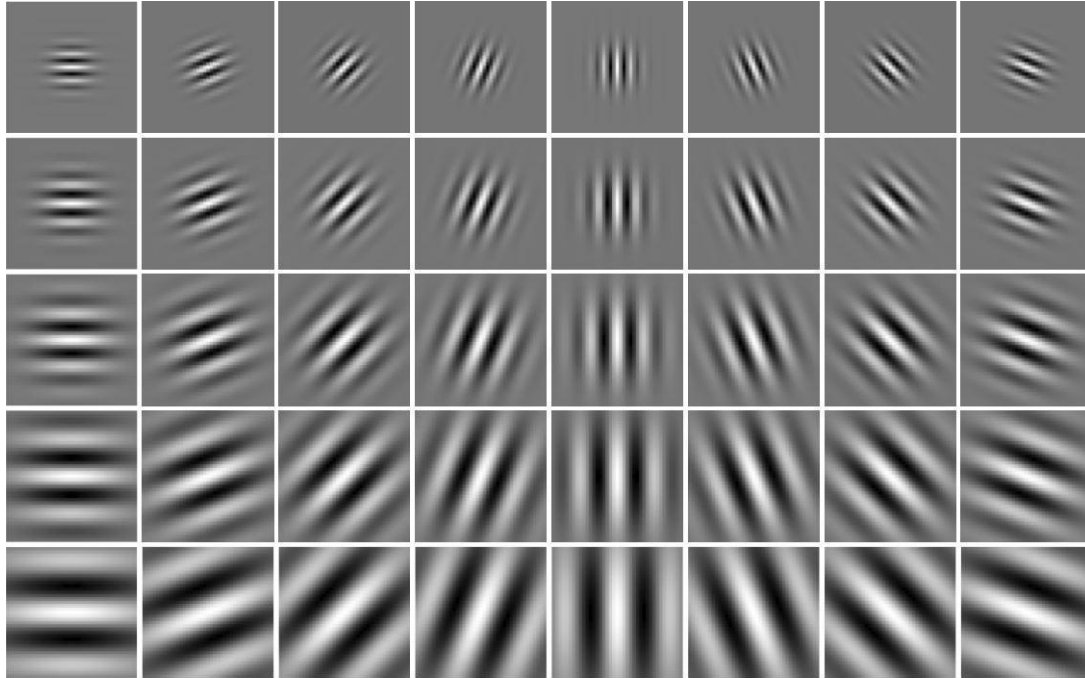
# Gabor Filters



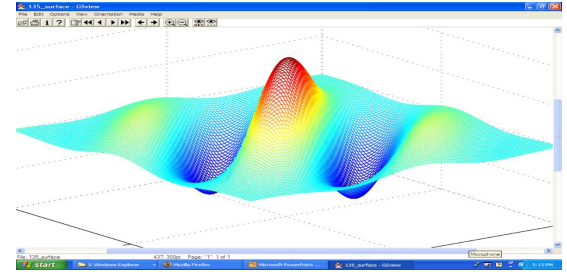
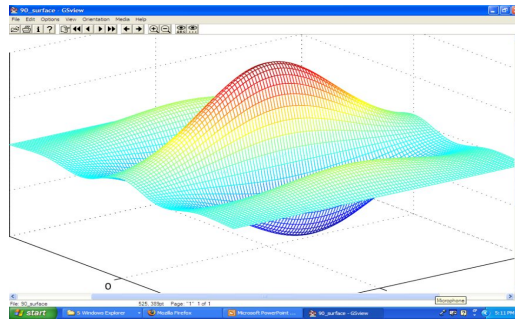
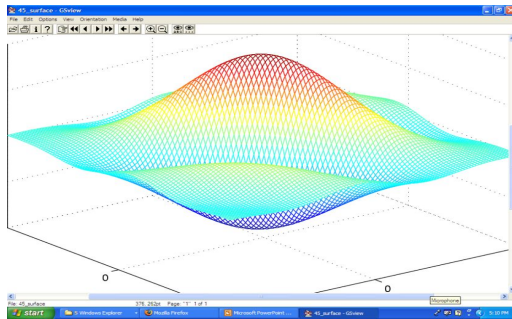
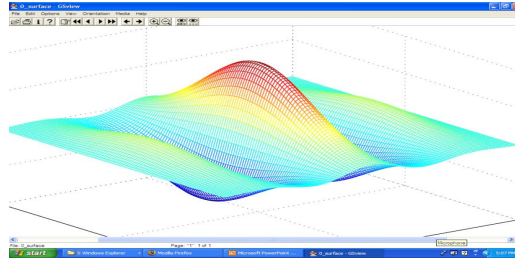
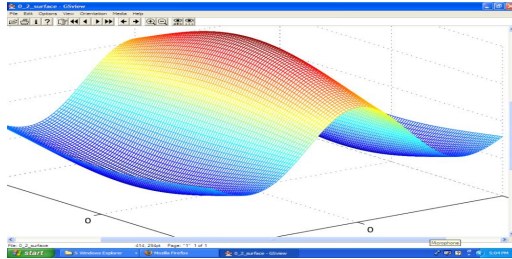
# Gabor Filters

## Set of convolution kernels

Different frequencies and orientations



# Gabor Filters Convolution kernels – in 3D visualisation

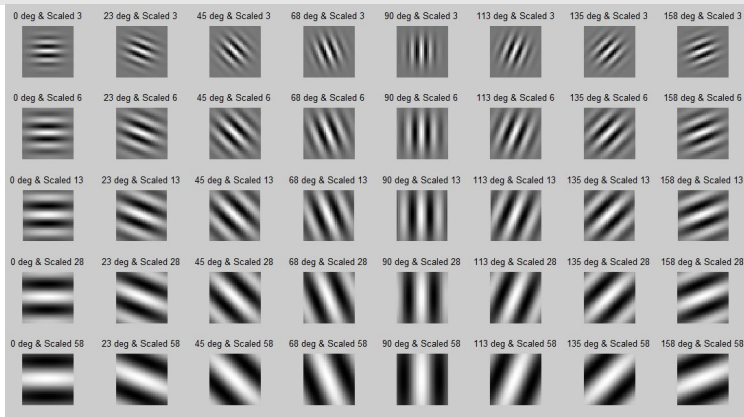


# Gabor filters – example

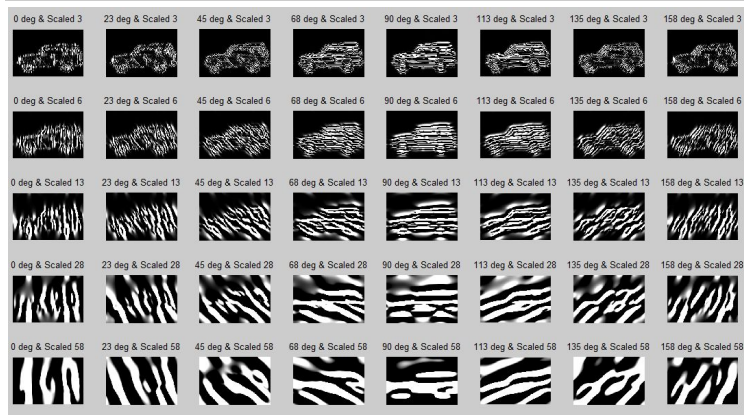


Input image

Gabor kernels



Convolution output



Zheng, Danian, Yannan Zhao, and Jiaxin Wang. "Features extraction using a gabor filter family." *Proceedings of the sixth Lasted International conference, Signal and Image processing, Hawaii*. 2004.

# Gabor filters example

Input image



Orientation=0, Wavelength=2



Orientation=0, Wavelength=4



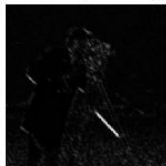
Orientation=0, Wavelength=6



Orientation=0, Wavelength=8



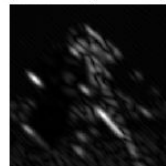
Orientation=45, Wavelength=2



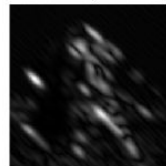
Orientation=45, Wavelength=4



Orientation=45, Wavelength=6



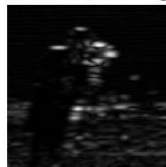
Orientation=45, Wavelength=8



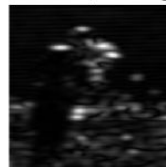
Orientation=90, Wavelength=2



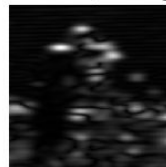
Orientation=90, Wavelength=4



Orientation=90, Wavelength=6



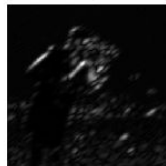
Orientation=90, Wavelength=8



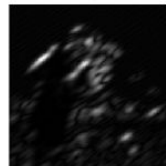
Orientation=125, Wavelength=2



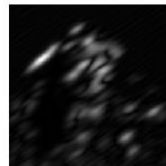
Orientation=125, Wavelength=4



Orientation=125, Wavelength=6



Orientation=125, Wavelength=8



# Convolution in Neural Networks

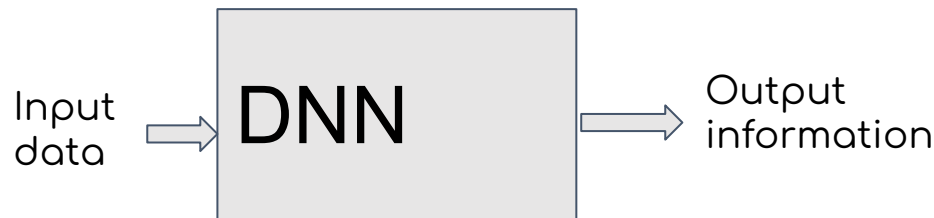
# Type of tasks solved by DNN

## Analytical tasks

Classification

Segmentation

Regression

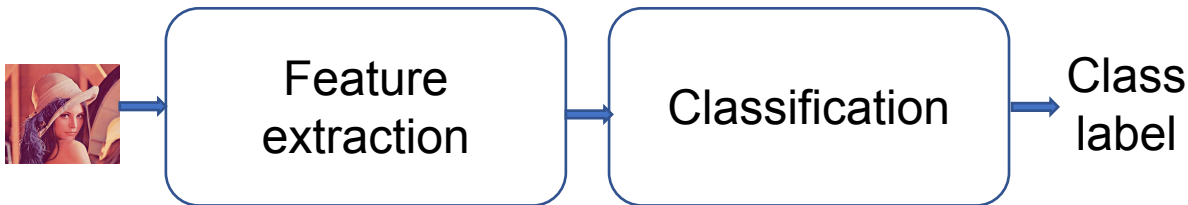


## Generative tasks



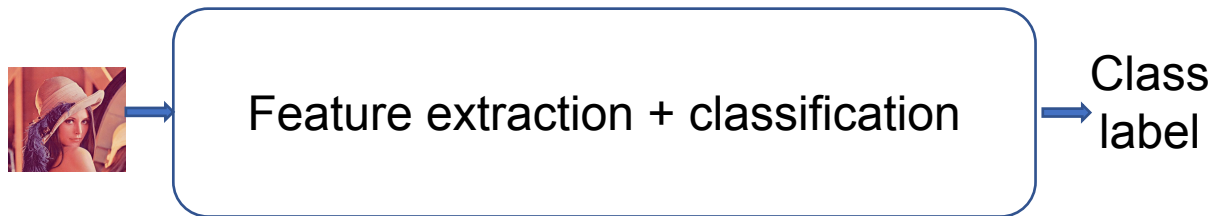
+ Combination of all

# Classification task and Deep Learning



- **Traditional approach**

Suboptimal,  
requires expert knowledge,  
hand crafted features



- **Deep learning**

both the classifiers and the  
features are learned  
automatically

# Image Classification - Object recognition



[This image is CC0 public domain](#)

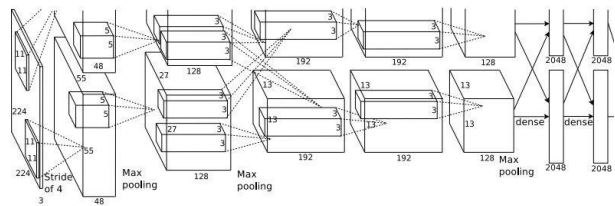


Figure copyright Alex Krizhevsky, Ilya Sutskever, and Geoffrey Hinton, 2012. Reproduced with permission.

**Vector:**  
4096

→  
**Fully-Connected:**  
4096 to 1000

**Class Scores**

Cat: 0.9

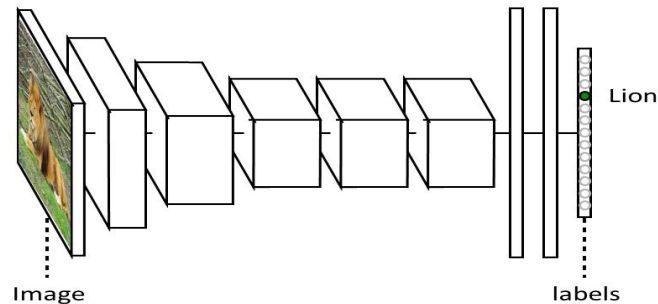
Dog: 0.05

Car: 0.01

...

## Alex Net

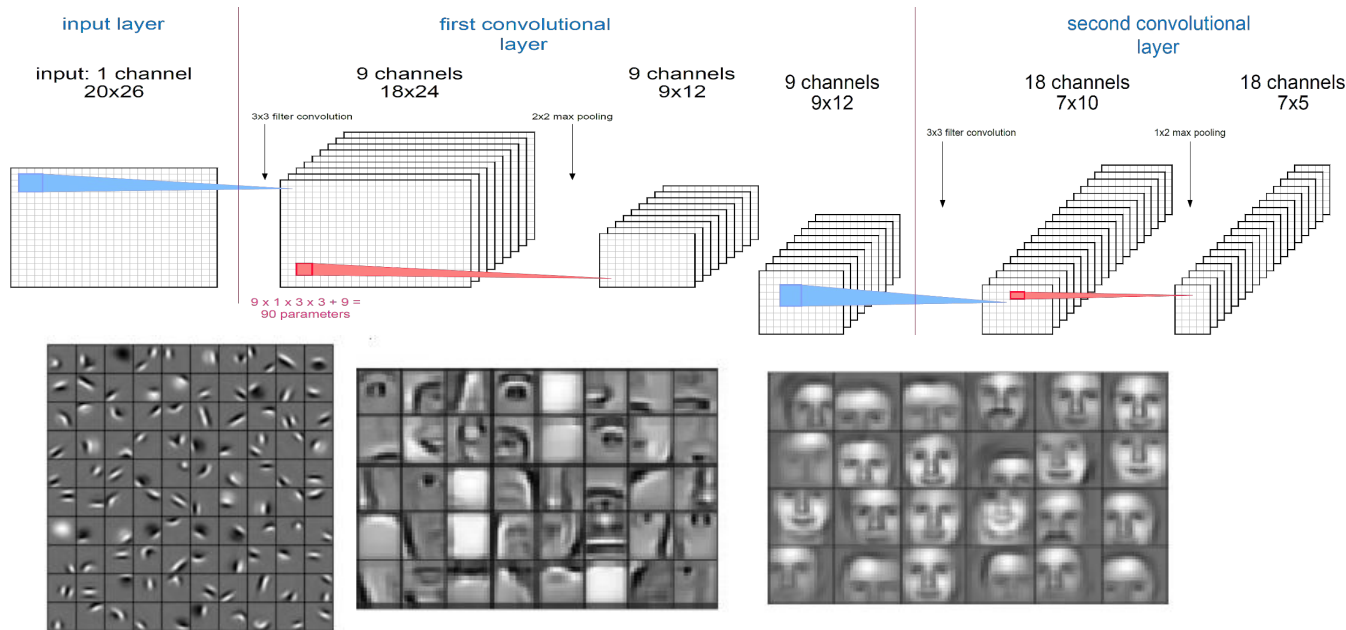
Convolutional Neural Networks: AlexNet



Krizhevsky, Sutskever, Hinton — NIPS 2012

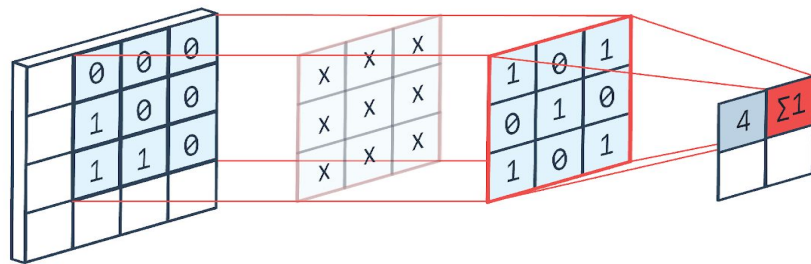
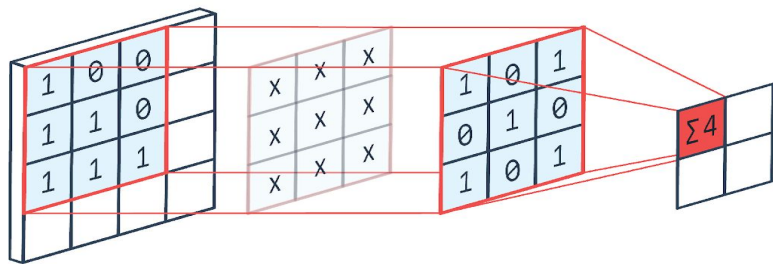
slide credit Jason Yosinski

# Convolutional layers in CNN - example



Learned hierarchical features from a deep learning algorithm.

# 2D Convolution block – Neural Networks



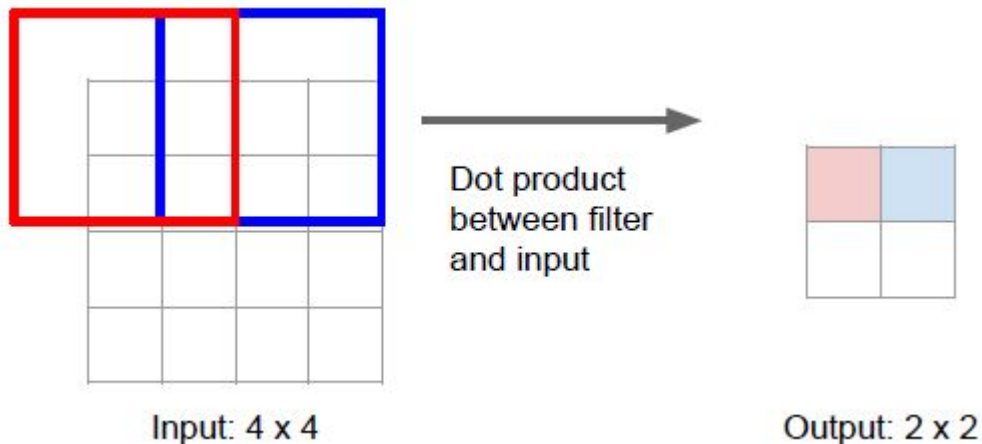
- each layer is composed by a configurable **number of filters**
- each filter is matrix of **trainable weights**
- convolution operation is performed between the image and **each filter**, producing as **output a new image (tensor)**
- Every value in the tensor is then fed through an **activation function** to introduce a nonlinearity ( applied principles of artificial neuron)

# Parameters of Convolution filters

- **Number of filters:** The number of convolutional filters in the layer
- **Width of filter:** The width of the filter matrix of a single filter.
- **Height of filter:** The width of the weight matrix of a single filter.
- **Horizontal stride:** The number of pixels to move while performing the convolution along the horizontal axis.
- **Vertical stride:** The number of pixels to move while performing the convolution along the vertical axis.
- **Padding:** Same results in padding the input such that the output has the same length as the original input.
- **Activation function:** The function that will applied to each element of the output. (ReLU, sigmoidal...)

# Strided convolution

example: 3 x 3 convolution, stride 2 pad 1



Filter moves 2 pixels in the input for every one pixel in the output

Stride gives ratio between movement in input and output

# ConvNetJS demo: training on CIFAR-10

## ConvNetJS CIFAR-10 demo

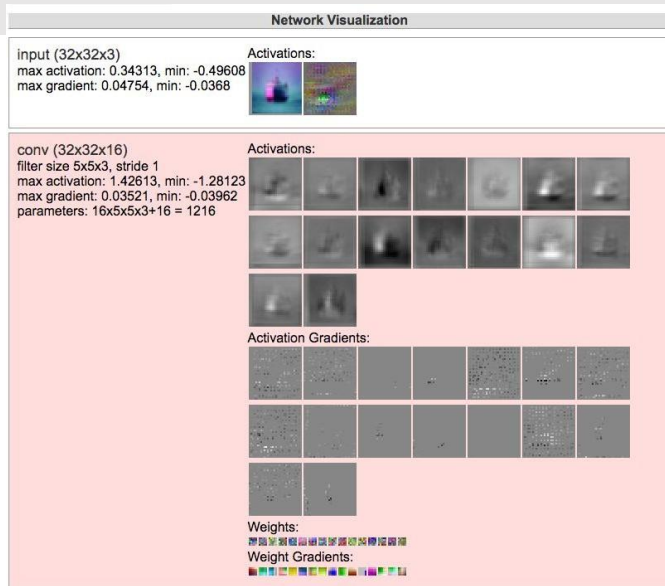
### Description

This demo trains a Convolutional Neural Network on the [CIFAR-10 dataset](#) in your browser, with nothing but Javascript. The state of the art on this dataset is about 90% accuracy and human performance is at about 94% (not perfect as the dataset can be a bit ambiguous). I used [this python script](#) to parse the [original files](#) (python version) into batches of images that can be easily loaded into page DOM with img tags.

This dataset is more difficult and it takes longer to train a network. Data augmentation includes random flipping and random image shifts by up to 2px horizontally and vertically.

By default, in this demo we're using Adadelata which is one of per-parameter adaptive step size methods, so we don't have to worry about changing learning rates or momentum over time. However, I still included the text fields for changing these if you'd like to play around with SGD+Momentum trainer.

Report questions/bugs/suggestions to [@karpathy](mailto:@karpathy).



<http://cs.stanford.edu/people/karpathy/convnetjs/demo/cifar10.html>

Image gradient vector

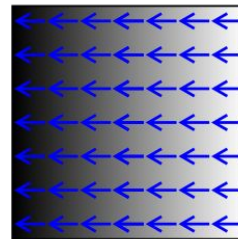
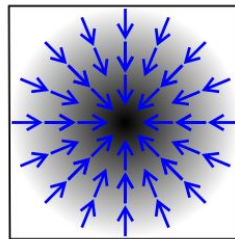
# Image gradient vector

- gradient vector of the image intensity
- vector at each point in the image

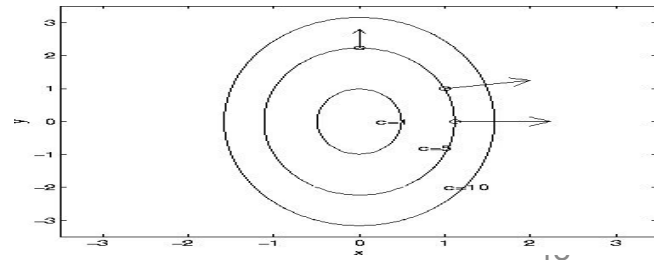
The gradient of an image is given by the formula:

$$\nabla I = \left( \frac{\partial I}{\partial x}, \frac{\partial I}{\partial y} \right)$$

image gradient  
(example):



edge filtered image  
(example):



# Image gradient vector

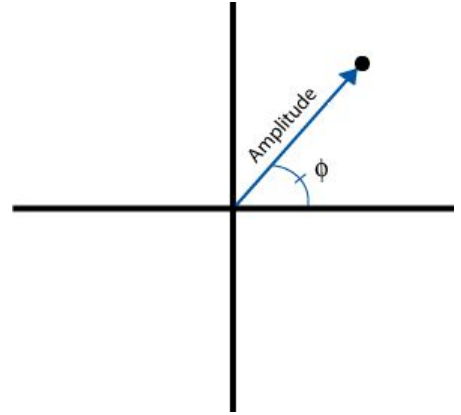
Calculates the gradient vector of the image intensity at each point using Prewitt (Sobel) filter

Magnitude  
(amplitude)

$$G = \sqrt{G_x^2 + G_y^2},$$

angle

$$\Theta = \arctan(G_y, G_x)$$



# Image gradient - example

Sobel filter - edge horizontal



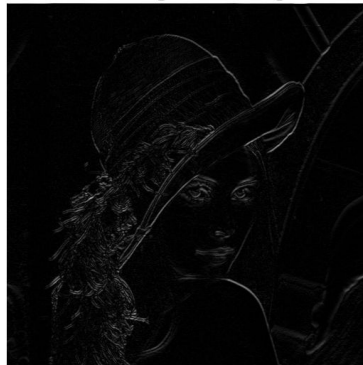
$G_x$

Magnitude  
(amplitude)

$$G = \sqrt{G_x^2 + G_y^2},$$



Sobel filter - gradient magnitude



$G_y$

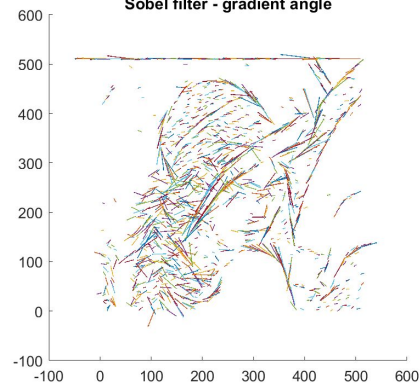


Angle  
(visualisation of  $\theta_{xy}$  :

$$\Theta = \arctan(G_y, G_x)$$



Sobel filter - gradient angle



# Sharpening

# Sharpening - example



# Sharpening using Spatial Filters

the principal objective of sharpening is

- to highlight fine details (edges) in an image
- or to enhance fine details that have been blurred, either disturbed
- or as a natural effect of a particular method of image acquisition.

# Sharpening approaches for creation of the edge image

## 1. Edge image

Using blur of the original image :

Subtract the blurred image from the original

Using Laplacian image:

Laplacian edge image from the original image

..other edge image....

## 2. Add/subtract the edge image from the original image

# Sharpening 1D example

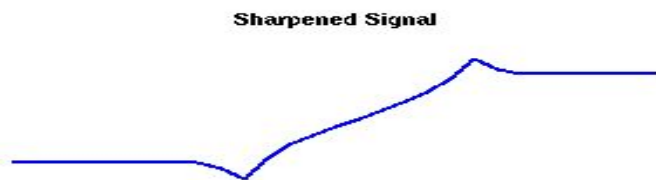
Original signal  $S_o$ :



Blurred signal  $S_b$



$Dif = S_o - S_b$



# Sharpening by Laplacian

Add/Subtract the original and Laplacian images

If the definition of Laplacian filter kernel used has a **negative center** coefficient, then we **subtract the image**

|   |    |   |   |    |   |
|---|----|---|---|----|---|
| 0 | 1  | 0 | 1 | 1  | 1 |
| 1 | -4 | 1 | 1 | -8 | 1 |
| 0 | 1  | 0 | 1 | 1  | 1 |

If the definition of Laplacian filter kernel used has a **positive center** coefficient, then we **add the image**

|    |    |    |    |    |    |
|----|----|----|----|----|----|
| 0  | -1 | 0  | -1 | -1 | -1 |
| -1 | 4  | -1 | -1 | 8  | -1 |
| 0  | -1 | 0  | -1 | -1 | -1 |

$$g(x, y) = \begin{cases} f(x, y) - \nabla^2 f(x, y) & \text{if the center coefficient of the Laplacian mask is negative} \\ f(x, y) + \nabla^2 f(x, y) & \text{if the center coefficient of the Laplacian mask is positive.} \end{cases}$$

# Sharpening

Original image + Laplacian Image

|   |   |   |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 0 |
| 0 | 0 | 0 |

 + 

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

Sharpening kernel:

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 9  | -1 |
| -1 | -1 | -1 |

Convolution properties

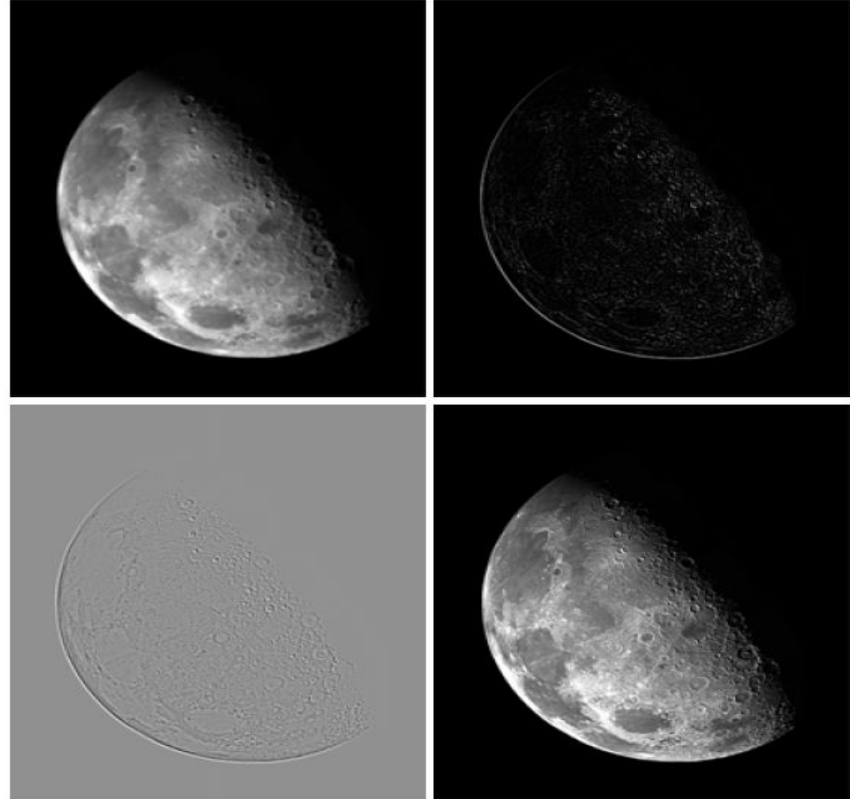
Distributive:

$$f[x,y] * (g_1[x,y] + g_2[x,y]) = f[x,y] * g_1[x,y] + f[x,y] * g_2[x,y]$$

# Example of sharpening using Laplacian

1. Image of the North Pole of the moon.
1. Laplacian filtered image.
1. Laplacian image rescaled (scaled for display purposes.)
1. Image enhanced by using sharpening:

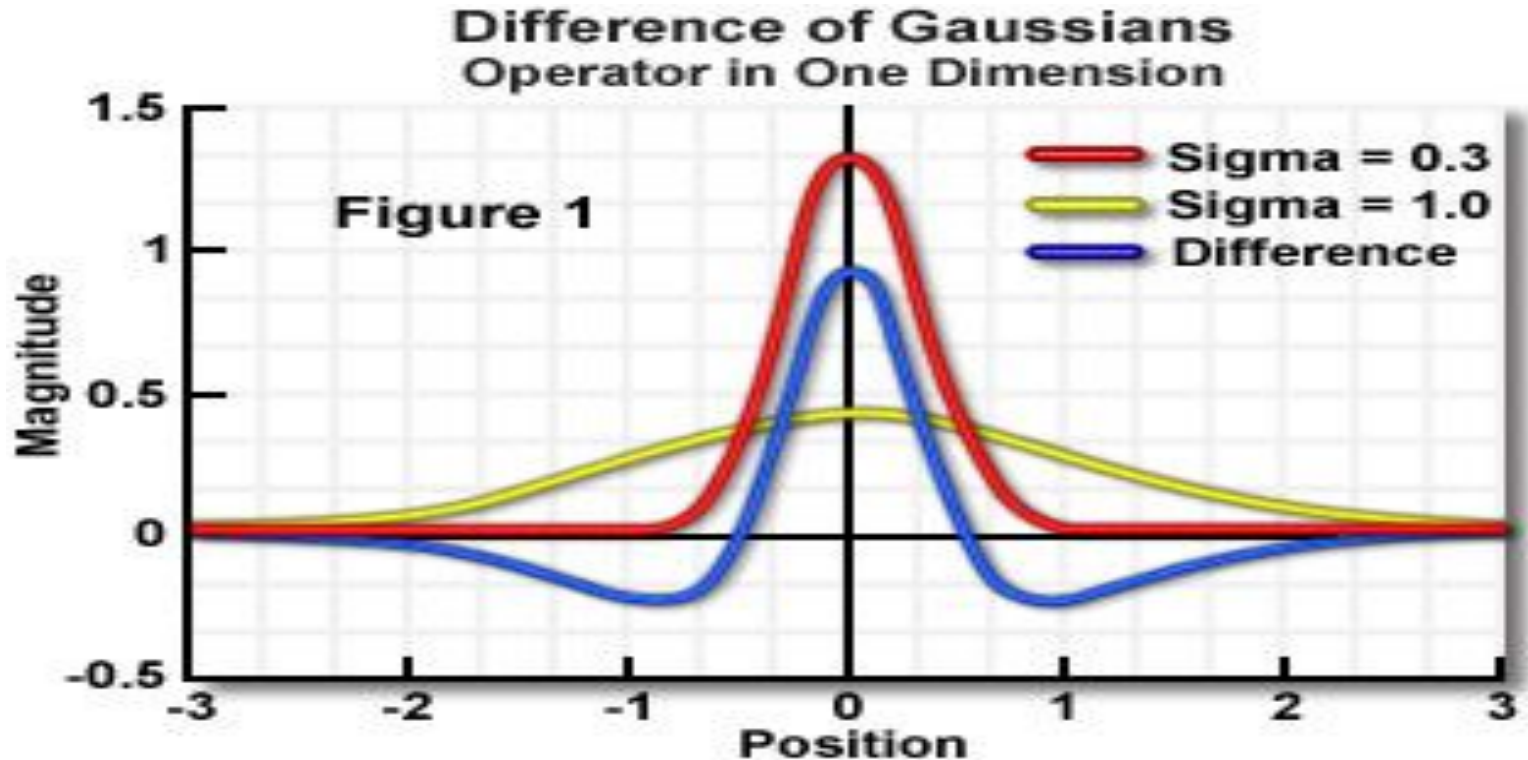
Original image + Laplacian Image



Edge detection  
...a lot of methods...

# Difference of Gaussians - DoG

## Edge detection



# Difference of Gaussians (DoG)

## Edge detection - example



Input image



IF2=Gaussian filter  $\sigma = 2$

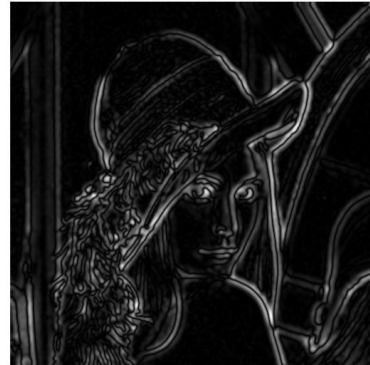


IF5=Gaussian filter  $\sigma = 5$

IF2-IF5



abs(IF2-IF5)



# Laplacian of Gaussian LoG

## Combine Smoothing with Differentiation

Laplacian of Gaussian (LoG) Filter - useful for finding edges

Laplacian of Gaussian: Marr&Hildrith have proposed an **operator** in which Gaussian shaped smoothing is performed prior to the application of the Laplacian

LoG filtering = Gaussian smoothing + Laplacian filtering

# Laplacian of Gaussian - LoG

## Combine Smoothing with Differentiation

commonly used discrete approximations of Laplacian filter:

|    |    |    |
|----|----|----|
| 0  | -1 | 0  |
| -1 | 4  | -1 |
| 0  | -1 | 0  |

|    |    |    |
|----|----|----|
| -1 | -1 | -1 |
| -1 | 8  | -1 |
| -1 | -1 | -1 |

these kernels are approximating a second derivative measurement on the image->they are very sensitive to noise.

-> the image is Gaussian smoothed before applying the Laplacian filter.

# Laplacian of Gaussian - LoG

## Combine Smoothing with Differentiation

Solution:

the convolution operation is associative  $f * (g_1 * g_2) = (f * g_1) * g_2$

->we can convolve the Gaussian smoothing filter with the Laplacian filter,

and then convolve this hybrid filter with the image.

advantage:

- The LoG (Laplacian of Gaussian) kernel can be pre-calculated in advance so only one convolution needs to be performed at run-time on the image.

$$f_{\text{laplacian\_filter}} * (f_{\text{gauss\_filter}} * \text{image}) = (f_{\text{laplacian\_filter}} * f_{\text{gauss\_filter}}) * \text{image}$$

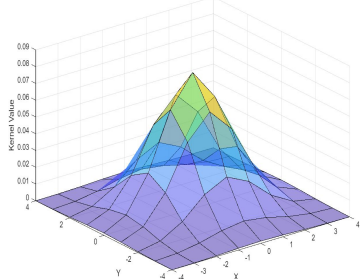
# Laplacian of Gaussian - LoG

|   |    |   |
|---|----|---|
| 0 | 1  | 0 |
| 1 | -4 | 1 |
| 0 | 1  | 0 |

\*

|     |     |     |     |     |     |     |     |     |
|-----|-----|-----|-----|-----|-----|-----|-----|-----|
| 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.1 | 0.1 | 0.1 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.1 | 0.3 | 0.4 | 0.3 | 0.1 | 0.0 | 0.0 |
| 0.0 | 0.1 | 0.3 | 0.6 | 0.8 | 0.6 | 0.3 | 0.1 | 0.0 |
| 0.0 | 0.1 | 0.4 | 0.8 | 1.0 | 0.8 | 0.4 | 0.1 | 0.0 |
| 0.0 | 0.1 | 0.3 | 0.6 | 0.8 | 0.6 | 0.3 | 0.1 | 0.0 |
| 0.0 | 0.0 | 0.1 | 0.3 | 0.4 | 0.3 | 0.1 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.1 | 0.1 | 0.1 | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 | 0.0 |

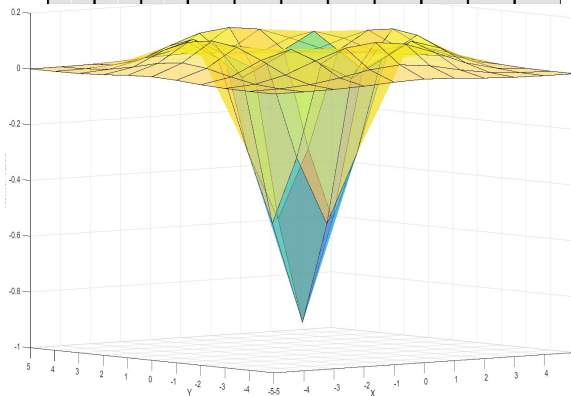
3D Plot of 9x9 Gaussian Kernel ( $\sigma = 1.4$ )



Gaussian kernel.

=

|     |     |     |      |      |      |      |      |     |     |     |
|-----|-----|-----|------|------|------|------|------|-----|-----|-----|
| 0.0 | 0.0 | 0.0 | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0  | 0.0  | 0.1  | 0.0  | 0.0  | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.1  | 0.1  | 0.1  | 0.1  | 0.1  | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.1 | 0.1  | 0.1  | -0.0 | 0.1  | 0.1  | 0.1 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.1 | 0.1  | -0.3 | -0.5 | -0.3 | 0.1  | 0.1 | 0.0 | 0.0 |
| 0.0 | 0.1 | 0.1 | -0.0 | -0.5 | -0.9 | -0.5 | -0.0 | 0.1 | 0.1 | 0.0 |
| 0.0 | 0.0 | 0.1 | 0.1  | -0.3 | -0.5 | -0.3 | 0.1  | 0.1 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.1 | 0.1  | 0.1  | -0.0 | 0.1  | 0.1  | 0.1 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.1  | 0.1  | 0.1  | 0.1  | 0.1  | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0  | 0.0  | 0.1  | 0.0  | 0.0  | 0.0 | 0.0 | 0.0 |
| 0.0 | 0.0 | 0.0 | 0.0  | 0.0  | 0.0  | 0.0  | 0.0  | 0.0 | 0.0 | 0.0 |



The 2-D  
Laplacian of  
Gaussian  
(LoG)  
kernel.

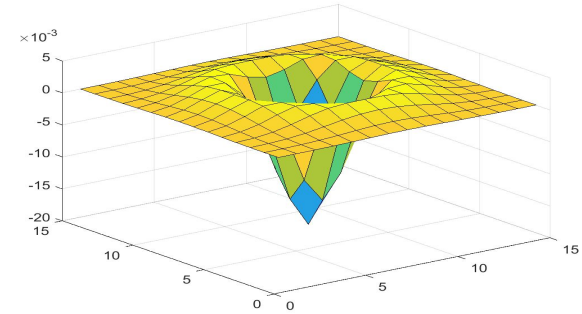
Laplacian  
kernel.

# Example of LoG



Input image

LoG Filter Kernel  
Size = 15,  
Sigma=2



LoG filtered Image



abs(LoG filtered Image)

# Advanced methods

## Adaptive filtration

# Adaptive filtration

Adaptive filtration applies a linear filter to an image adaptively, tailoring itself to the local image features (variance...etc)

Example:

Where the variance is large, then performs little smoothing. Where the variance is small, then performs more smoothing.

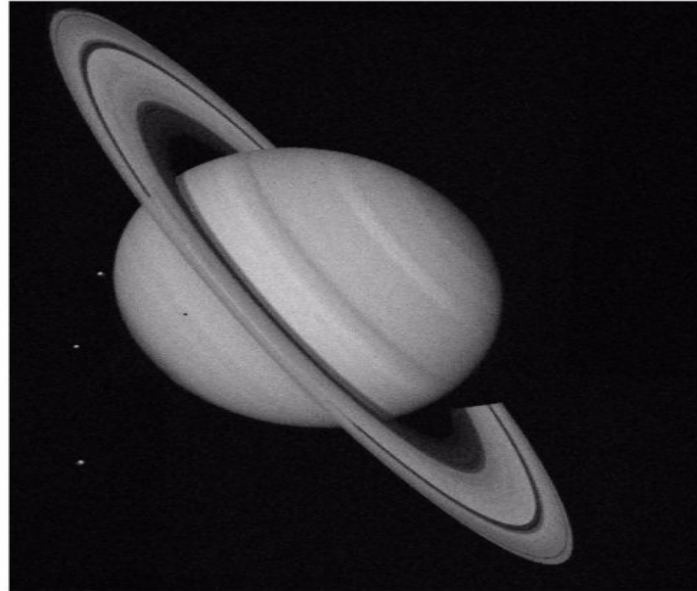
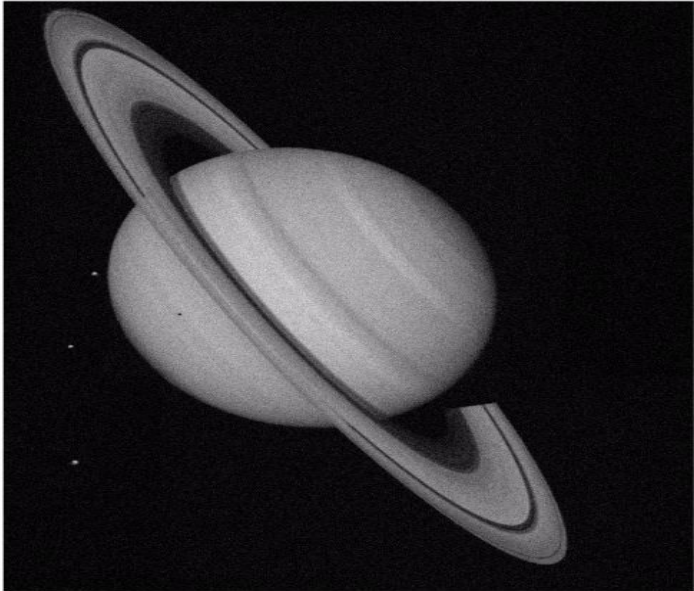
# Adaptive filtration

This approach often produces better results than linear filtering.

The adaptive filter is more selective than a comparable non-adaptive filter, preserving edges and other high-frequency parts of an image.

# Example of adaptive filtration

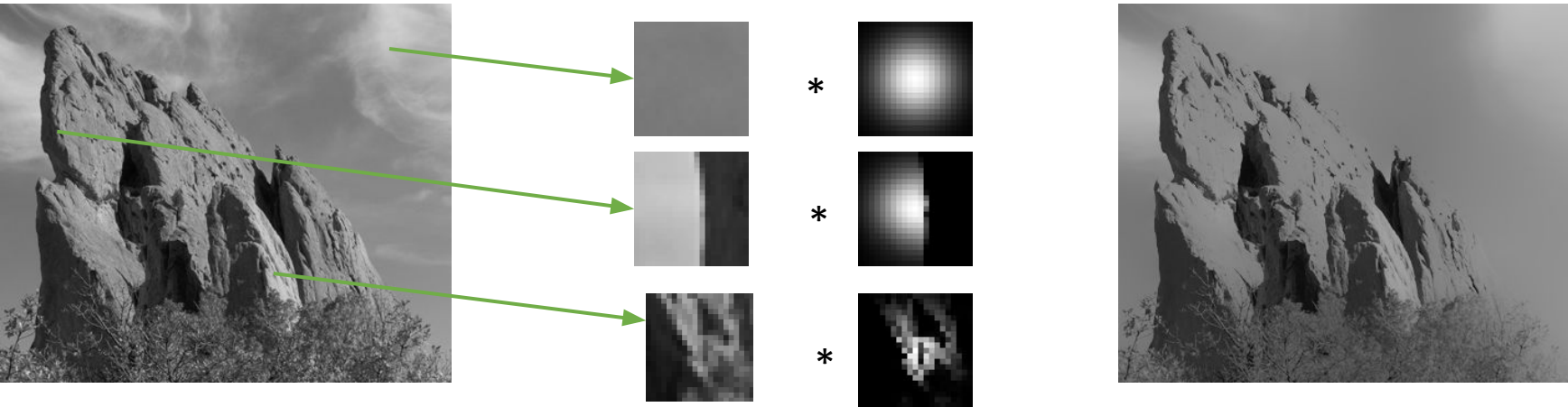
The example below applies adaptive filtration using the Wiener filter to an image of Saturn that has had Gaussian noise added.



# Bilateral Filter

## No Averaging across Edges - Bilateral Edges

The kernel shape depends on the image content.



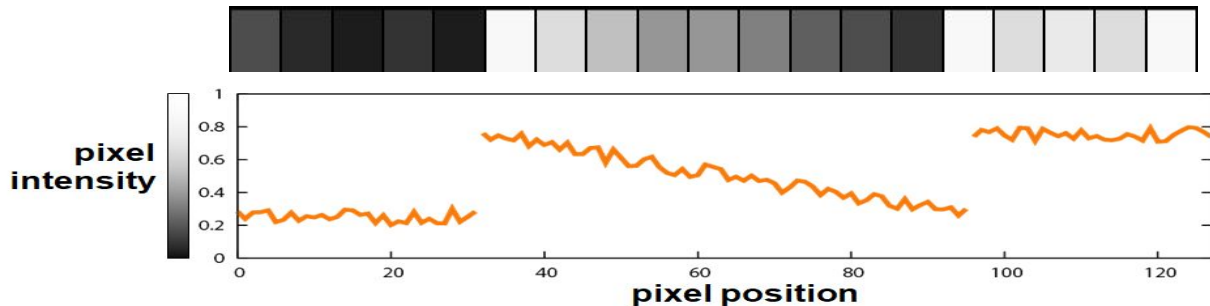
Edge preserving

# Bilateral filtering

## Illustration a line of image

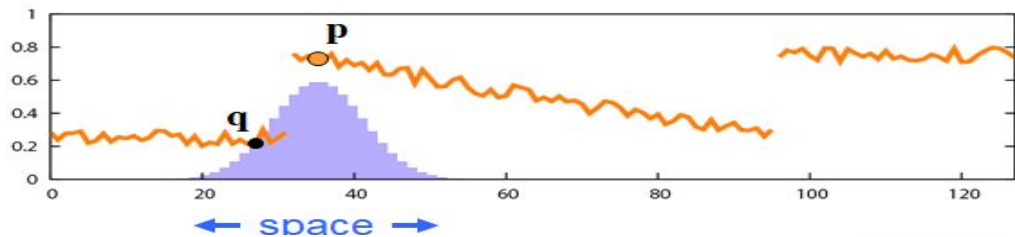
line of pixels

visualisation as a plot



## Gaussian blur filter

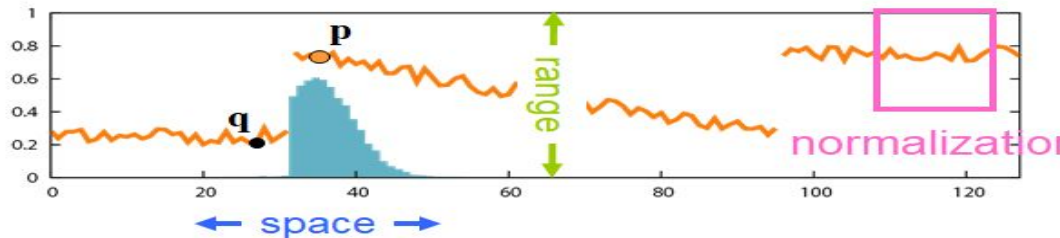
- only spatial distance, intensity ignored



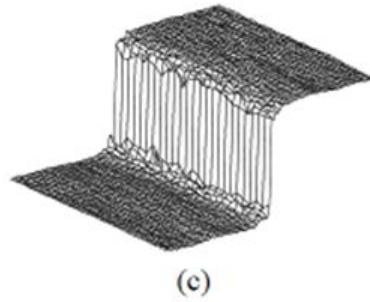
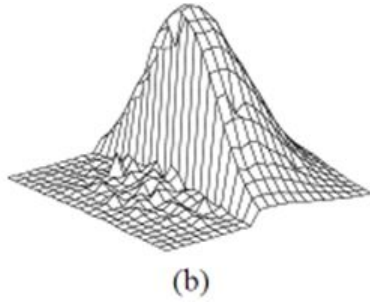
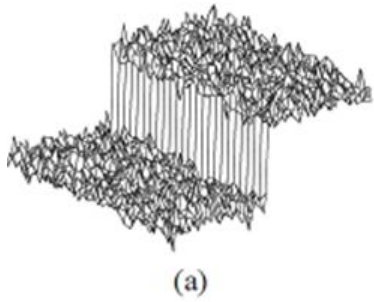
## Bilateral filter

[Aurich 95, Smith 97, Tomasi 98]

- spatial and range distances
- weights sum to 1



# Bilateral filtering



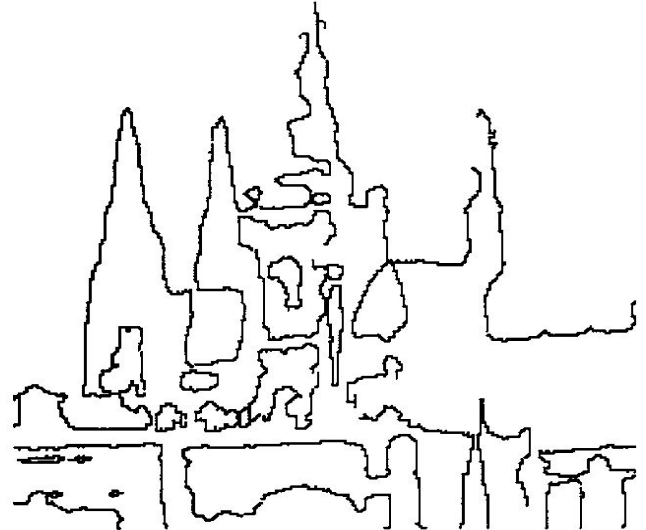
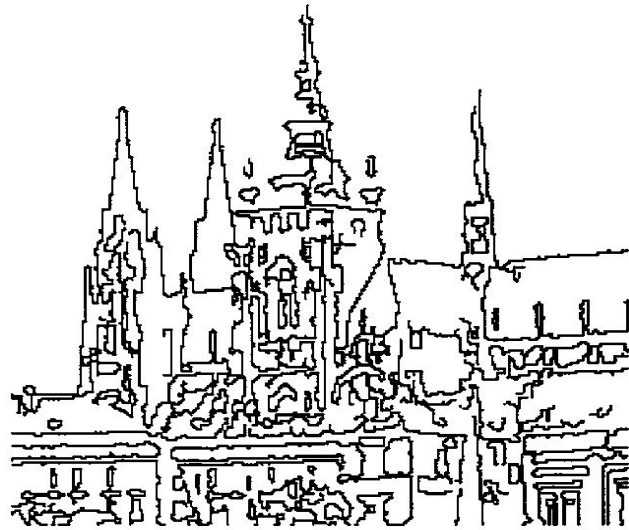
Combined domain and range filtering will be denoted as bilateral filtering.

# Canny detector

# Canny edge detection -> binary contours

Example - Canny edge detection at two different scales

[Hlavac, Sonka, Boyle]



# Canny edge detection Algorithm

## 1. Noise Reduction – Gaussian blur

Convolve an image  $f$  with a Gaussian ( std  $\sigma$ )

## 2. Finding Intensity Gradient of the Image

Estimate local edge normal directions for each pixel in the image.

- Smoothened image is then filtered with a Sobel kernel in both horizontal and vertical direction to get first derivative in horizontal direction () and vertical direction (). From these two images, we can find edge gradient and direction for each pixel.
- See „image gradient“

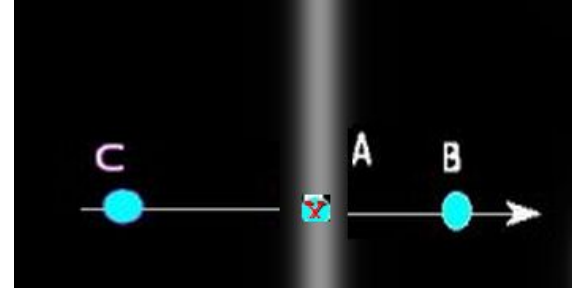
# Canny edge detection

## Algorithm – cont.

### 3. Non-maximum Suppression

Eliminate points that do not lie in important edges - find the location of the edges using non-maximal suppression.

- Point A is on the edge ( in vertical direction). Gradient direction is normal to the edge. Point B and C are in gradient directions. So point A is checked with point B and C to see if it forms a **local maximum**. If so, it is considered for next stage, otherwise, it is suppressed ( put to zero).



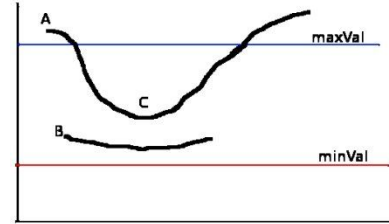
-> Compute the magnitude of the edge

# Canny edge detection Algorithm – cont.

## 4. Hysteresis Thresholding

Threshold edges in the image with hysteresis to eliminate spurious responses.

- Parameters: **two threshold** values,  $\text{minVal}$  and  $\text{maxVal}$ .
- Any edges with intensity gradient **more than  $\text{maxVal}$**  are sure to be edges
- and those edges **below  $\text{minVal}$**  are sure to be **non-edges** -> discarded.
- Those who lie between these two thresholds are classified edges or non-edges based on their connectivity. If they are connected to “sure-edge” pixels, they are considered to be part of edges. Otherwise, they are also discarded.



# Canny edge detection

## Algorithm – cont.

5. Repeat steps (1) through (4) for ascending values of the standard deviation  $\sigma$

6. Aggregate the final information about edges at multiple scale using the 'feature synthesis' approach.

# Methods of noise reduction

# Removing of noise by Order filters

Implemented by arranging the neighborhood pixels in order from smallest to largest gray level value, and using this ordering to select the "correct" value

Order filters such as the median can be used to smooth images

Order filters work best with salt-and-pepper, negative exponential, or Rayleigh noise

! Salt-and-pepper noise could be efficiently removed using the median filter

# Image restoration using median filtration



# Removing of noise - Linear filters

Low-pass filters can be used to reduce noise effects

The linear filters work properly in the case of Gaussian or uniform noise

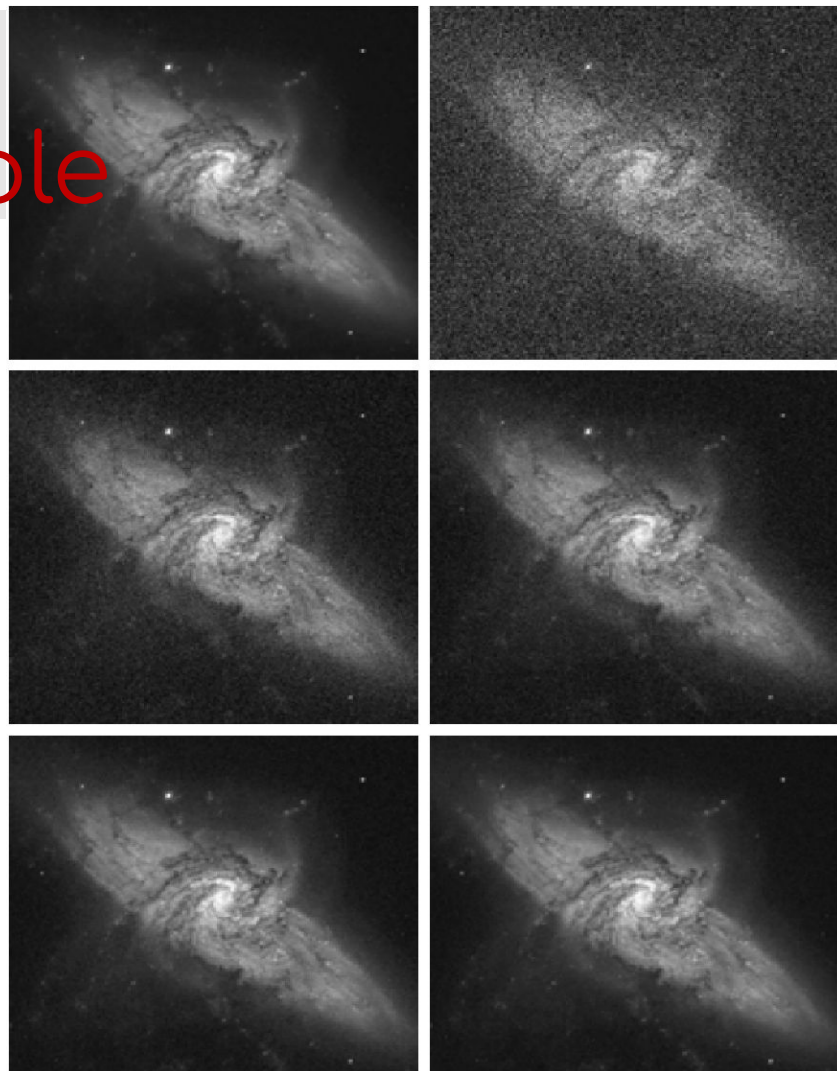
The linear filters have the disadvantage of blurring the image edges or details

# Removing of noise - Linear filters - example

(a) Image of Galaxy Pair NGC 3314.

(b) Image corrupted by additive Gaussian noise with zero mean and a standard deviation of 64 gray levels.

(c)-(f) Results of averaging the noisy images (mean filter)  
Size :  $K=8, 16, 64$ , and  $128$ .



# Denoising using Deep Neural Networks

Example: Stacked denoising autoencoder

